

SPRINT 3 - BRIEF 2

AffiliateLink: Plateforme d'affiliation

Contexte du brief

Vous construisez la première brique de **AffiliateLink**: le **module profile**. Il gère l'identité de tous les acteurs de la plateforme : chaque publisher et chaque advertiser y crée son compte, que l'administrateur valide avant qu'il puisse utiliser la plateforme. Toutes les autres fonctionnalités en dépendent, puisqu'aucune offre ni aucune affiliation ne peut exister sans un advertiser ou un publisher dont le compte a été validé.

Le code est dès maintenant organisé en module ; Spring Modulith et ses règles de vérification seront mis en place au brief suivant, lorsque le module offer devra communiquer avec le module profile.

Spécifications fonctionnelles

User stories

- En tant que publisher, je crée mon profil avec mon email, mon prénom, mon nom, ma niche, une courte bio et mon pays.
- En tant qu'advertiser, je crée mon compte avec mon email, la raison sociale de mon entreprise, mon secteur et mon site web.
- En tant qu'utilisateur, je consulte un profil de publisher ou d'advertiser par son identifiant.
- En tant qu'utilisateur, je consulte la liste des publishers et la liste des advertisers.
- En tant qu'administrateur, je valide, suspends ou réactive un compte.

Règles de gestion

- L'email est **unique sur toute la plateforme** : un même email ne peut pas servir à un publisher et à un advertiser.
- Les niches disponibles sont : `BEAUTY`, `TECH`, `GAMING`, `TRAVEL`, `LIFESTYLE`, `FOOD`, `SPORT`.
- Tout nouveau compte est créé avec le statut `PENDING`.
- Un compte n'est jamais supprimé : il est suspendu.
- Seules ces transitions de statut sont autorisées :

Statut actuel	Transition autorisée
`PENDING`	`ACTIVE`
`ACTIVE`	`SUSPENDED`
`SUSPENDED`	`ACTIVE`

Spécifications techniques

Entités

- **Profile** (abstraite) : id, email (unique), status, createdAt.
- **Publisher** (hérite de Profile) : firstName, lastName, niche, bio, country.
- **Advertiser** (hérite de Profile) : companyName, sector, website.
- Héritage JPA en stratégie **JOINED** : une table commune et une table par type de profil.

Endpoints

Méthode	URL	Réponse
POST	`/api/publishers`	201 / 400 / 409
GET	`/api/publishers`	200
GET	`/api/publishers/{id}`	200 / 404
POST	`/api/advertisers`	201 / 400 / 409
GET	`/api/advertisers`	200
GET	`/api/advertisers/{id}`	200 / 404
PATCH	`/api/admin/profiles/{id}/status`	204 / 404 / 409

Organisation du code

- Le code est organisé **par domaine** et non package-by-layer.
- Tout ce qui concerne les profils se trouve dans `com.affiliateLink.profile`, et les éléments communs (exceptions, gestion des erreurs) dans `com.affiliateLink.common`.
- Les entités, repositories, services, DTO et contrôleurs sont placés dans des sous-packages de `com.affiliateLink.profile`.

Exigences

- Architecture en couches : contrôleur, service, repository.
- Injection de dépendances par constructeur.
- **DTO sous forme de records** en entrée et en sortie : les entités ne sont jamais exposées par l'API.
- Validation des entrées avec Bean Validation (`@Valid`, `@NotBlank`, `@Email`…).
- Les règles métier (transitions de statut) sont portées par les entités, pas par les contrôleurs.
- Erreurs renvoyées via un `@RestControllerAdvice` : 400 pour les données invalides, 404 pour une ressource introuvable, 409 pour une règle de gestion non respectée.
- Schéma de la base créé et versionné avec **Flyway** ; Hibernate en mode `ddl-auto: validate`.
- Tables préfixées par le nom du module (`profile_profiles`, `profile_publishers`…).
- PostgreSQL lancé avec **Docker Compose**, démarré automatiquement par Spring Boot.